

数字组招生实践项目

A / B 二选一题目说明

本次考核提供两个方向，参与者选择其中一道完成，不要求同时完成两道题。两道题都以数字系统设计、RTL 仿真和工程汇报为主要训练目标。

A 题：Attention 加速器 IP 设计与验证

围绕 Q、K、V、Score、Softmax 和加权求和，完成一个基础 Attention RTL IP 的设计、仿真和验证。具体要求见后续 A 题正文。

B 题：基于图像处理的仿真验证平台

搭建 FPGA 图像处理仿真平台，将输入图像存入 RAM，读取 RAM 中的像素数据进行处理，并以 BMP 格式输出处理结果。具体方向包括颜色识别、形态学处理、AI 算子移植、双目摄像头数据拼接与处理等。具体要求见后续 B 题正文。

请根据个人兴趣和基础选择一个方向。最终提交内容、实现深度和汇报重点以所选题目的正文要求为准。

从公式到 RTL：Attention 加速器 IP 设计与验证

大学集成电路学院数字组招生实践项目 · 面向大二学生 · 建议周期 1-2 周

项目说明

本项目不要求参与者事先掌握 Transformer、Attention 或 AI 加速器。题目提供必要背景、公式、接口建议和计算示例。参与者需要通过自主学习，把给定算法转化为一个可综合、可仿真的数字 IP，并在最终汇报中说明自己的设计。

考核形式：自主学习与实现 1-2 周，提交 RTL、验证代码、报告和演示材料，随后统一进行现场演示与答辩。

一、项目背景：为什么要做 Attention 加速

Transformer 是当前大模型中常见的网络结构，已经应用于文本生成、机器翻译、视觉理解等任务。Attention 的作用可以直观地理解为：当前 Token 通过 Q 去查询一组历史 Token，用相似程度决定应该从哪些历史信息中读取更多内容。

对于一个 Query 来说，系统需要让它依次与很多个 Key 做点积，再把得到的分数转换成权重，最后对对应的 Value 做加权求和。当历史序列长度从几十增加到几百、几千甚至更长时，Q 与 K 的匹配次数以及权重与 V 的乘加次数都会随之增加。若用矩阵形式一次性计算，还需要保存大量中间 Score 或 Probability 数据。

这带来两个硬件问题。第一，乘法器和累加器要处理大量重复计算；第二，片上 SRAM 容量有限，不能无限保存中间矩阵，数据往往需要在片上 Buffer

与片外存储之间搬运。片外存储访问通常比片上寄存器或 SRAM 慢，也会消耗更多带宽和能量，因此 Attention 不只是“算得多”，还存在明显的访存瓶颈。

本项目把问题限定在一个很小的 Attention

计算单元中，使学生能够先理解基本数学过程，再把它翻译成数字电路。对于本题的

$S=8$ 、 $d=4$ ，允许使用片上寄存器或 Buffer 保存 8 个 Score 和 8 个权重，不要求额外实现大模型中的存储优化方法。本项目不要求复现任何特定论文或完整大模型，而是完成最基本、最适合入门的 Attention 硬件 IP。

本题最终完成的不是完整 SoC，而是一个可以进一步封装进 SoC 的 Attention 加速器 RTL IP 原型。

二、项目设计目标

1. 正确性目标：根据给定的 Q、K、V 输入，正确计算 Score、权重、L、ACC 和最终输出，并用 Python Golden Model 对 RTL 结果进行逐项验证。
2. 数据通路目标：按照清晰的阶段完成 Q/K/V 装载、Score 计算、权重计算、加权累加和归一化。允许保存本题规模下的中间 Score 和权重，重点是理解数据如何在寄存器、Buffer 和计算单元之间流动。
3. 硬件实现目标：使用寄存器、Buffer、乘法器、加法器、累加器、查找表和有限状态机完成计算，使每个模块的职责和数据流向都能解释清楚，并具备综合条件。
4. 时序验证目标：明确 start、valid、ready、done 和输出数据之间的时序关系，能够通过仿真波形说明每个时钟周期正在处理什么数据。
5. 性能分析目标：统计计算周期、资源使用和存储需求，并讨论当序列长度 S 从 8 扩展到 256 时，计算量、Buffer 容量和外部带宽需求如何变化。

三、最少背景知识

1. Q、K、V 是什么

可以把一次 Attention 理解为“当前信息去查询历史信息”。Q（Query）表示当前查询；每个 K（Key）用于判断一条历史信息与 Q 的相关程度；对应的 V（Value）是这条历史信息真正携带的内容。

名称	直观含义	本题中的形状
Q	当前想查询什么	[1, d]
K	每个 Token 的匹配特征	[S, d]
V	每个 Token 携带的内容	[S, d]
O	按相关性加权得到的输出	[1, d]

2. Attention 的三步计算

第一步，计算 Q 与每个 K 的点积，得到相关性分数：

$$\text{score}_j = Q \cdot K_j = \sum Q[k] \times K_j[k]$$

第二步，Softmax 将 Score 转成非负权重。为方便硬件实现，本题提供指数查找表：

$$\begin{aligned} \text{weight}_j &= \exp(\text{score}_j) \\ L &= \sum \text{weight}_j \end{aligned}$$

第三步，使用权重对 V 加权求和并归一化：

$$\begin{aligned} \text{ACC} &= \sum \text{weight}_j \times V_j \\ O &= \text{ACC} / L \end{aligned}$$

三、完整手算示例

为便于手算，本例使用 $S=2$ 、 $d=2$ ，并用题目给定的近似指数表代替真实指数函数。

```
Q = [1, 2]
K0 = [1, 0] K1 = [0, 1]
V0 = [4, 2] V1 = [1, 6]
```

步骤 1：计算 Score

```
score0 = 1×1 + 2×0 = 1
score1 = 1×0 + 2×1 = 2
```

步骤 2：查指数表并累计分母

假设题目给出 $\exp(1)=3$ 、 $\exp(2)=7$ ，则 $\text{weight0}=3$ 、 $\text{weight1}=7$ ， $L=3+7=10$ 。

步骤 3：计算分子向量 ACC

```
ACC = 3×[4, 2] + 7×[1, 6]
ACC = [12, 6] + [7, 42] = [19, 48]
```

步骤 4：归一化得到输出

```
O = ACC/L = [19/10, 48/10] = [1.9, 4.8]
```

硬件不会一次完成整条公式，而是用多个时钟周期逐步完成乘法、加法、查表和累加。

四、正式任务规模

项目	固定要求
Batch / Head	1 / 1
Token 数量 S	8
Head 维度 d	4
输入	$Q : [1, 4]$; $K : [8, 4]$; $V : [8, 4]$
输出	$O : [1, 4]$
基础数据格式	8-bit 有符号整数；中间结果必须使用更宽位宽
指数与倒数	由出题方提供 LUT 或参考模块

五、硬件计算流

建议的数据流如下。参与者可以调整内部结构，但必须保证功能正确，并能解释修改原因。

输入阶段：Q、K、V 通过输入接口进入 Q 寄存器和 K/V Buffer。

Score 阶段：点积单元依次读取 Q 和 K0 到 K7，计算 8 个 Score，并保存到 Score Buffer。

权重阶段：依次读取 Score Buffer 中的 Score，访问指数查找表，得到 8 个 weight，并保存到 Weight Buffer。

累加阶段：读取 Weight Buffer 和 V Buffer，计算 L，并将 8 组 $\text{weight} \times V$ 累加到 ACC。

归一化阶段：8 个 K/V 全部处理完成后，计算 $O = \text{ACC} / L$ 。

输出阶段：依次输出 O[0] 到 O[3]，最后产生 done。

本题只有一个 Query，规模固定为 $S=8$ ，因此可以直接保存 8 个 Score 和 8 个权重。题目重点是基础 Attention 的计算与 RTL 验证，不考察大规模矩阵的存储优化。

六、参考时序与状态机

以下时序只是参考，不要求完全一致。学生可以使用更多或更少周期，但必须满足接口协议。

阶段	参考动作	主要寄存器/模块
IDLE	等待 start	状态机
LOAD_Q	连续 4 拍装载 Q	Q-Reg
LOAD_KV	装载 K、V，或边装载边计算	K/V Buffer
DOT	4 次乘法累加得到一个 score	乘法器、Score 累加器
EXP	根据 score 访问指数 LUT	exp_lut
UPDATE	更新 L 和 4 路 ACC	L、ACC[0:3]
NEXT	处理下一个 K/V，共 8 组	Token 计数器
NORM	计算 ACC/L	倒数 LUT 或除法器
OUTPUT	依次输出 4 个结果	out_valid、done

建议先画状态转移图，再写 RTL。若先写大量 always 块，通常很难处理计数器清零、valid 对齐和连续任务。

七、建议接口

基础版本不要求 AXI 和 DMA。出题方提供顶层模板，接口可采用以下形式：

```
module attention_ip (...);
input clk, rst_n, start;
input signed [7:0] q_in, k_in, v_in;
input q_valid, kv_valid;
output data_ready;
output signed [15:0] out_data;
output out_valid, done;
endmodule
```

接口约定

- start 仅在空闲状态有效，拉高 1 拍开始一次任务。
- q_valid 表示 q_in 有效，共输入 4 个元素。
- kv_valid 表示当前 k_in/v_in 有效，按 K0、V0 到 K7、V7 顺序输入。
- data_ready 为 1 时才能发送输入。
- out_valid 为 1 时 out_data 有效，共输出 4 个元素。
- done 拉高 1 拍表示任务结束，之后模块返回 IDLE。

八、位宽提示

两个 8-bit 有符号数相乘至少产生 16-bit 结果。4 项点积累加还需要额外位宽； $\text{weight} \times V$ 和 ACC 也要防止溢出。参与者需要在报告中写出自己的位宽推导。直接把所有中间结果都截成 8 bit 会丢失大量信息。

九、分阶段任务

阶段	建议时间	完成内容
1. 背景学习	第 1-2 天	理解 Q/K/V、点积、Softmax、L、ACC，并完成手算题
2. 数据通路	第 2-4 天	画模块图、状态机和时序，确定位宽
3. 基础 RTL	第 4-7 天	完成装载、点积、L/ACC 累加和输出
4. 验证调试	第 7-10 天	完成 directed/random tests，与 Golden Model 对比
5. 优化与汇报	剩余时间	完成加分项、周期分析、报告和演示

十、必须提交的内容

- 完整 RTL 源代码。
- 自己编写或扩展的 testbench。
- 仿真脚本、波形和结果截图。
- 5-8 页设计报告。
- 5-8 分钟汇报 PPT。
- README：说明工具版本、运行命令和文件结构。

报告至少说明：公式理解、整体数据流、模块划分、状态机、位宽、验证方法、周期数、遇到的问题和改进方向。

十一、基础验收

验收项	要求
编译	RTL 和 testbench 可在指定仿真器中编译
功能	通过公开样例和隐藏随机样例
时序	reset、start、valid、ready、done 符合约定
结构	Score、Weight、L、ACC 等中间数据结构清晰，位宽合理
可解释性	能够在现场说明数据流和关键代码

十二、进阶加分项

参与者可任选，不要求全部完成：

- A. 因果 Mask：当 $j > i$ 时屏蔽对应 Key。
- B. 流水线或并行优化：减少周期并报告资源代价。
- C. 参数化：支持可配置 S 或 d。
- D. 简单筛选：输出大于动态阈值的 Key 索引，作为稀疏 Attention 扩展。

加分项必须附独立测试。仅写在报告中但没有可运行代码和结果，不计功能加分。

十三、现场汇报与答辩

建议每人 5-8 分钟汇报，随后 5 分钟问答和现场演示。评委可随机更换一组输入，要求学生运行仿真并解释波形。

答辩不是考记忆，而是核实学生是否真正理解并完成了设计。允许使用开发工具和资料，但必须能解释代码、测试与结果。

十四、边界说明

基础题不要求大规模模型、AXI4、DMA、DDR 控制器、完整 GPT-2、FP16 运算、UVM、Cadence

物理综合、布局布线或真实功耗测量。学有余力者可以在报告中讨论更复杂的 Attention 优化，但不能替代基础功能。

项目最终产物：一个功能正确、可综合、可仿真、可解释的基础 Attention RTL IP 原型。

题目三：基于图像处理的仿真验证平台

基本：

自行搭建FPGA图像处理系统仿真平台，输入一张图像之后，将图像存储到RAM里面，然后通过读取RAM的像素数据对RAM进行处理，最后能将图像处理后的图像以bmp格式输出。请实现多种图像处理算法，要求可以在testbench中通过宏定义选择不同的算法。

可选算法如下：

- 1. 颜色识别、目标检测算法等。
- 2. 缩放、膨胀、腐蚀、锐化、梯形校正、混叠等。
- 3. AI算子移植实现目标检测、手势识别等。
- 4. 双目摄像头数据拼接与处理。

算法类型	算法名称
基础算法	直方图均衡化、图像缩放（固定倍数）、图像旋转（固定角度）、形态学操作、颜色空间转换等基础类算法
进阶算法	双边滤波、导引滤波、仿射变换、暗光处理、高动态范围、脸部特效（磨皮美化、局部形变等）、物品特效（背景替换分割、物品材质替换等）、手势特效（手势识别、虚拟物品操控等）等进阶类算法